

Deep Learning Laboratory Manual

Practical Implementations using Keras, TensorFlow, and PyTorch

Experiments 1 – 10

Table of Contents

Table of Contents	2
Experiment 1 — Multi-Layer Perceptron for MNIST Handwritten Digit Classification	3
Experiment 2 — Binary Classification of Movie Reviews using IMDB Dataset	6
Experiment 3 — Multi-Class Classification of Newswires using Reuters Dataset	9
Experiment 4 — Predicting House Prices using Boston Housing Dataset	12
Experiment 5 — Build a Convolutional Neural Network for MNIST Digit Classification.....	14
Experiment 6 — CNN for Simple Image Classification (Cats vs Dogs)	17
Experiment 7 — Use a Pre-Trained Convolutional Neural Network (VGG16) for Image Classification.....	19
Experiment 9 — Word Embeddings for IMDB Dataset.....	22
Experiment 10 — Implement a Recurrent Neural Network (RNN) for IMDB Movie Review Classification.....	25

Experiment 1 — Multi-Layer Perceptron for MNIST Handwritten Digit Classification

Aim: To implement a Multi-Layer Perceptron (MLP) using Keras, TensorFlow, and PyTorch for MNIST digit classification.

Software Requirements

- Python 3.x
- Keras
- TensorFlow
- PyTorch
- NumPy, Matplotlib

Theory

The MNIST dataset contains 70,000 grayscale images of handwritten digits (0–9). MLPs are feedforward neural networks that map input data to output classes using multiple dense (fully connected) layers.

Implementation

A. Using Keras

```
from keras.datasets import mnist
from keras.models import Sequential
from keras.layers import Dense, Flatten
from keras.utils import to_categorical

# Load dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Preprocess
x_train, x_test = x_train / 255.0, x_test / 255.0
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)

# Model
model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.fit(x_train, y_train, epochs=5, batch_size=32)
loss, acc = model.evaluate(x_test, y_test)
print(f'Test Accuracy: {acc:.2f}')
```

B. Using TensorFlow

```

import tensorflow as tf

mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0

model = tf.keras.models.Sequential([
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(x_train, y_train, epochs=5)
print(model.evaluate(x_test, y_test))

```

C. Using PyTorch

```

import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms

transform = transforms.ToTensor()
train_set = torchvision.datasets.MNIST(root='./data', train=True,
                                       download=True, transform=transform)
test_set = torchvision.datasets.MNIST(root='./data', train=False,
                                       download=True, transform=transform)

train_loader = torch.utils.data.DataLoader(train_set, batch_size=32, shuffle=True)
test_loader = torch.utils.data.DataLoader(test_set, batch_size=32, shuffle=False)

class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(28 * 28, 128)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = x.view(-1, 28 * 28)
        return self.fc2(self.relu(self.fc1(x)))

model = MLP()
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

for epoch in range(5):
    for images, labels in train_loader:
        outputs = model(images)
        loss = criterion(outputs, labels)

```

```
optimizer.zero_grad()  
loss.backward()  
optimizer.step()
```

Sample Output

```
Final Accuracy ~ 97.5% (Keras)  
Final Accuracy ~ 98%   (TensorFlow)  
Final Accuracy ~ 97%   (PyTorch)
```

Result: Successfully implemented an MLP using Keras, TensorFlow, and PyTorch for MNIST digit classification.

Viva Questions

1. What is a perceptron?
2. Explain activation functions used in MLP.
3. Why is ReLU used instead of sigmoid?
4. What is the use of softmax?
5. What is the MNIST dataset?

Experiment 2 — Binary Classification of Movie Reviews using IMDB Dataset

Aim: To build a neural network for binary sentiment classification (positive or negative) using the IMDB dataset with Keras, TensorFlow, and PyTorch.

Software Requirements

- Python 3.x
- Keras
- TensorFlow
- PyTorch
- NumPy, Matplotlib

Theory

The IMDB dataset consists of 50,000 movie reviews labeled as positive or negative. A binary classifier is trained to determine sentiment using word embeddings and dense neural networks.

Implementation

A. Using Keras

```
from keras.datasets import imdb
from keras.models import Sequential
from keras.layers import Dense, Embedding, GlobalAveragePooling1D
from keras.preprocessing.sequence import pad_sequences

vocab_size = 10000
max_len = 300

(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=vocab_size)
x_train = pad_sequences(x_train, maxlen=max_len)
x_test = pad_sequences(x_test, maxlen=max_len)

model = Sequential([
    Embedding(vocab_size, 32, input_length=max_len),
    GlobalAveragePooling1D(),
    Dense(16, activation='relu'),
    Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

model.fit(x_train, y_train, epochs=5, batch_size=32)
loss, acc = model.evaluate(x_test, y_test)
print(f'Test Accuracy: {acc:.2f}')
```

B. Using TensorFlow

```
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

```

vocab_size = 10000
max_len = 300

(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=vocab_size)
x_train = pad_sequences(x_train, maxlen=max_len)
x_test = pad_sequences(x_test, maxlen=max_len)

model = tf.keras.Sequential([
    tf.keras.layers.Embedding(vocab_size, 32),
    tf.keras.layers.GlobalAveragePooling1D(),
    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

model.fit(x_train, y_train, epochs=5)
model.evaluate(x_test, y_test)

```

C. Using PyTorch

```

import torch
import torch.nn as nn
from torchtext.datasets import IMDB
from torchtext.data.utils import get_tokenizer
from torchtext.vocab import build_vocab_from_iterator
from torch.utils.data import DataLoader
from torchtext.data.functional import to_map_style_dataset

# Placeholder: PyTorch IMDB requires preprocessing or HuggingFace
# datasets (recommended for real usage)
print("Please use HuggingFace Datasets or torchtext with manual "
      "preprocessing for real implementation.")

```

Sample Output

Test Accuracy ~ 87-89%

Result: Successfully implemented a binary classification model for IMDB reviews using word embeddings and dense layers.

Viva Questions with Answers

1. What is sentiment analysis?

Sentiment analysis is the process of identifying and categorizing opinions expressed in a piece of text as positive, negative, or neutral.

2. What does the IMDB dataset contain?

It contains 50,000 movie reviews labeled for binary sentiment classification.

3. Why do we use padding in NLP?

To ensure all sequences are of the same length for batch processing in neural networks.

4. What is an embedding layer?

It converts words (represented by integers) into dense vectors of fixed size.

5. Why is sigmoid used in the final layer?

Because it outputs probabilities between 0 and 1, ideal for binary classification.

Experiment 3 — Multi-Class Classification of Newswires using Reuters Dataset

Aim: To design a neural network for classifying news articles into multiple categories using the Reuters dataset with Keras, TensorFlow, and PyTorch.

Software Requirements

- Python 3.x
- Keras
- TensorFlow
- PyTorch
- NumPy, Matplotlib

Theory

The Reuters dataset contains 11,228 news articles from Reuters, labeled over 46 categories. This is a classic multi-class classification task. We use an embedding layer and dense layers with a softmax output to classify the articles.

Implementation

A. Using Keras

```
from keras.datasets import reuters
from keras.models import Sequential
from keras.layers import Dense, Embedding, GlobalAveragePooling1D
from keras.preprocessing.sequence import pad_sequences
from keras.utils import to_categorical

vocab_size = 10000
max_len = 200

(x_train, y_train), (x_test, y_test) = reuters.load_data(num_words=vocab_size)
x_train = pad_sequences(x_train, maxlen=max_len)
x_test = pad_sequences(x_test, maxlen=max_len)
y_train = to_categorical(y_train, num_classes=46)
y_test = to_categorical(y_test, num_classes=46)

model = Sequential([
    Embedding(vocab_size, 64, input_length=max_len),
    GlobalAveragePooling1D(),
    Dense(64, activation='relu'),
    Dense(46, activation='softmax')
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.fit(x_train, y_train, epochs=8, batch_size=32)
loss, acc = model.evaluate(x_test, y_test)
print(f'Test Accuracy: {acc:.2f}')
```

B. Using TensorFlow

```
import tensorflow as tf
from tensorflow.keras.datasets import reuters
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical

vocab_size = 10000
max_len = 200

(x_train, y_train), (x_test, y_test) = reuters.load_data(num_words=vocab_size)
x_train = pad_sequences(x_train, maxlen=max_len)
x_test = pad_sequences(x_test, maxlen=max_len)
y_train = to_categorical(y_train, 46)
y_test = to_categorical(y_test, 46)

model = tf.keras.models.Sequential([
    tf.keras.layers.Embedding(vocab_size, 64, input_length=max_len),
    tf.keras.layers.GlobalAveragePooling1D(),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(46, activation='softmax')
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.fit(x_train, y_train, epochs=8)
print(model.evaluate(x_test, y_test))
```

C. Using PyTorch

```
# Reuters is not available directly in PyTorch datasets.
# Use HuggingFace Datasets or manual CSVs. A PyTorch implementation requires:
#   - DataLoader from CSV or HuggingFace
#   - Custom tokenizer
#   - EmbeddingBag or RNN with softmax output
```

Sample Output

```
Test Accuracy ~ 78-80%
Loss decreases over epochs
46-category classification with softmax
```

Result: Successfully implemented a multi-class classifier using the Reuters dataset, achieving ~80% accuracy.

Viva Questions with Answers

1. What is the Reuters dataset?

A dataset of 11,000+ news articles classified into 46 categories.

2. Why is softmax used in multi-class classification?

Softmax converts output into probabilities summing to 1, ideal for multiple class predictions.

3. What is categorical crossentropy?

A loss function used when labels are one-hot encoded and there are more than two classes.

4. What is one-hot encoding?

A technique to convert categorical labels into binary vectors.

5. Why do we use GlobalAveragePooling1D?

It reduces the output of the embedding layer to a single vector by averaging, reducing model complexity.

Experiment 4 — Predicting House Prices using Boston Housing Dataset

Aim: To build a neural network model that predicts house prices based on various features using the Boston Housing dataset.

Software Requirements

- Python 3.x
- Keras / TensorFlow
- NumPy, Matplotlib, Scikit-learn

Theory

The Boston Housing dataset contains 13 features (e.g., crime rate, number of rooms) and a target value: median house price. This is a regression problem where the output is a continuous value.

Implementation

A. Using Keras

```
from keras.datasets import boston_housing
from keras.models import Sequential
from keras.layers import Dense
from sklearn.preprocessing import StandardScaler

(x_train, y_train), (x_test, y_test) = boston_housing.load_data()

scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)

model = Sequential([
    Dense(64, activation='relu', input_shape=(13,)),
    Dense(64, activation='relu'),
    Dense(1)
])

model.compile(optimizer='adam', loss='mse', metrics=['mae'])
model.fit(x_train, y_train, epochs=100, batch_size=16, verbose=0)
loss, mae = model.evaluate(x_test, y_test)
print(f'Mean Absolute Error: {mae:.2f}')
```

Sample Output

```
Mean Absolute Error (MAE): ~2.1
```

Result: Successfully built a regression model for the Boston Housing dataset, achieving low error.

Viva Questions with Answers

1. What kind of problem is housing price prediction?

Regression.

2. Why use MAE instead of accuracy?

Because the output is continuous, not categorical.

3. What preprocessing technique is used?

Feature standardization (scaling).

4. What does the last layer return?

A single predicted price.

5. Which loss function is used?

Mean Squared Error (MSE).

Experiment 5 — Build a Convolutional Neural Network for MNIST Digit Classification

Aim: To build a CNN for recognizing handwritten digits from the MNIST dataset.

Theory

CNNs are specialized for processing grid-like data such as images. They automatically learn spatial hierarchies of features using convolutional layers, pooling layers, and fully connected layers.

Implementation

Keras

```
from keras.datasets import mnist
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from keras.utils import to_categorical

(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train.reshape(-1, 28, 28, 1) / 255.0
x_test = x_test.reshape(-1, 28, 28, 1) / 255.0
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)

model = Sequential([
    Conv2D(32, kernel_size=3, activation='relu', input_shape=(28, 28, 1)),
    MaxPooling2D(pool_size=2),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.fit(x_train, y_train, epochs=5, batch_size=64)
model.evaluate(x_test, y_test)
```

TensorFlow

```
import tensorflow as tf

mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train[..., tf.newaxis] / 255.0
x_test = x_test[..., tf.newaxis] / 255.0

model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32, 3, activation='relu', input_shape=(28, 28, 1)),
    tf.keras.layers.MaxPooling2D(2),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
```

```

        tf.keras.layers.Dense(10, activation='softmax')
    ])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(x_train, y_train, epochs=5)
model.evaluate(x_test, y_test)

```

PyTorch

```

import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms

train_loader = torch.utils.data.DataLoader(
    torchvision.datasets.MNIST('./data', train=True, download=True,
                               transform=transforms.ToTensor()),
    batch_size=64, shuffle=True)
test_loader = torch.utils.data.DataLoader(
    torchvision.datasets.MNIST('./data', train=False, download=True,
                               transform=transforms.ToTensor()),
    batch_size=64, shuffle=False)

class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(1, 32, 3), nn.ReLU(), nn.MaxPool2d(2)
        )
        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 13 * 13, 128), nn.ReLU(),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.conv(x)
        return self.fc(x)

model = CNN()
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

for epoch in range(5):
    for images, labels in train_loader:
        outputs = model(images)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

```

Sample Output

Test accuracy ~99% in 5 epochs.

Viva Questions with Answers

1. Why use convolutional layers?

To extract spatial features like edges, shapes, and textures.

2. What does the pooling layer do?

Reduces dimensionality and retains essential features.

3. Why is Flatten used before Dense layers?

To convert 2D feature maps into 1D vectors for classification.

4. Why use softmax in the last layer?

To convert scores into probabilities summing to 1.

Experiment 6 — CNN for Simple Image Classification (Cats vs Dogs)

Aim: To build a CNN that classifies images of cats and dogs.

Theory

This is a binary classification problem using a convolutional neural network. CNNs automatically learn features like edges, patterns, and object structures for classification.

Implementation

Keras Implementation

```
from keras.preprocessing.image import ImageDataGenerator
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(150, 150, 3)),
    MaxPooling2D(2, 2),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

train_datagen = ImageDataGenerator(rescale=1./255)
train_generator = train_datagen.flow_from_directory(
    'cats_and_dogs/train', target_size=(150, 150),
    batch_size=32, class_mode='binary')

model.fit(train_generator, epochs=5)
```

TensorFlow Implementation

```
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator

model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32, 3, activation='relu', input_shape=(150, 150, 3)),
    tf.keras.layers.MaxPooling2D(2),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

train_gen = ImageDataGenerator(rescale=1./255)
```

```

train_data = train_gen.flow_from_directory(
    'cats_and_dogs/train', target_size=(150, 150), class_mode='binary')

model.fit(train_data, epochs=5)

```

PyTorch Implementation

```

import torch
import torchvision.transforms as transforms
import torchvision.datasets as datasets
from torch.utils.data import DataLoader
import torch.nn as nn

transform = transforms.Compose([
    transforms.Resize((150, 150)),
    transforms.ToTensor()
])

train_data = datasets.ImageFolder('cats_and_dogs/train', transform=transform)
train_loader = DataLoader(train_data, batch_size=32, shuffle=True)

class CatDogCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(3, 32, 3), nn.ReLU(), nn.MaxPool2d(2),
            nn.Flatten(),
            nn.Linear(32 * 74 * 74, 128), nn.ReLU(),
            nn.Linear(128, 1), nn.Sigmoid()
        )

    def forward(self, x):
        return self.net(x)

model = CatDogCNN()
criterion = nn.BCELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

# Training loop omitted for brevity

```

Experiment 7 — Use a Pre-Trained Convolutional Neural Network (VGG16) for Image Classification

Aim: To use a pre-trained VGG16 model for classifying images, leveraging transfer learning to improve performance on small datasets.

Theory

VGG16 is a deep CNN trained on ImageNet (1.4M images, 1000 classes). In transfer learning, we reuse the convolutional base of VGG16 and train a new classifier for our specific dataset (e.g., cats vs dogs), saving training time and improving performance.

Implementation

Keras Implementation

```
from keras.applications import VGG16
from keras.models import Sequential
from keras.layers import Flatten, Dense
from keras.preprocessing.image import ImageDataGenerator

vgg_base = VGG16(weights='imagenet', include_top=False, input_shape=(150, 150, 3))

model = Sequential([
    vgg_base,
    Flatten(),
    Dense(256, activation='relu'),
    Dense(1, activation='sigmoid')
])

# Freeze VGG16 layers
vgg_base.trainable = False

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

datagen = ImageDataGenerator(rescale=1./255)
train_generator = datagen.flow_from_directory(
    'cats_and_dogs/train', target_size=(150, 150),
    batch_size=32, class_mode='binary')

history = model.fit(train_generator, epochs=5)
```

TensorFlow Implementation

```
import tensorflow as tf

base_model = tf.keras.applications.VGG16(include_top=False,
                                          weights='imagenet',
                                          input_shape=(150, 150, 3))

base_model.trainable = False

model = tf.keras.Sequential([
```

```

        base_model,
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(256, activation='relu'),
        tf.keras.layers.Dense(1, activation='sigmoid')
    ])

    model.compile(optimizer='adam',
                  loss='binary_crossentropy',
                  metrics=['accuracy'])

    train_gen = tf.keras.preprocessing.image.ImageDataGenerator(rescale=1./255)
    train_data = train_gen.flow_from_directory(
        'cats_and_dogs/train', target_size=(150, 150), class_mode='binary')

    history = model.fit(train_data, epochs=5)

```

PyTorch Implementation

```

import torch
import torchvision.models as models
import torchvision.transforms as transforms
import torchvision.datasets as datasets
from torch import nn

transform = transforms.Compose([
    transforms.Resize((150, 150)),
    transforms.ToTensor()
])

train_data = datasets.ImageFolder('cats_and_dogs/train', transform=transform)
train_loader = torch.utils.data.DataLoader(train_data, batch_size=32,
                                           shuffle=True)

vgg16 = models.vgg16(pretrained=True)

# Freeze pretrained layers
for param in vgg16.features.parameters():
    param.requires_grad = False

vgg16.classifier = nn.Sequential(
    nn.Linear(25088, 256),
    nn.ReLU(),
    nn.Dropout(0.5),
    nn.Linear(256, 1),
    nn.Sigmoid()
)

criterion = nn.BCELoss()
optimizer = torch.optim.Adam(vgg16.classifier.parameters(), lr=0.001)

# Training loop omitted for brevity

```

Training Accuracy Plot (Keras)

```
import matplotlib.pyplot as plt

plt.plot(history.history['accuracy'], label='Accuracy')
plt.title("Training Accuracy with VGG16")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.grid(True)
plt.legend()
plt.show()
```

Sample Output

```
Epoch 1/5 - 12s - loss: 0.4781 - accuracy: 0.7735
Epoch 2/5 - 11s - loss: 0.3559 - accuracy: 0.8440
Epoch 3/5 - 11s - loss: 0.3010 - accuracy: 0.8765
Epoch 4/5 - 11s - loss: 0.2608 - accuracy: 0.8975
Epoch 5/5 - 11s - loss: 0.2346 - accuracy: 0.9105
Final Accuracy: ~91%
```

Results

- The pre-trained VGG16 significantly boosts accuracy (~91%) compared to training a CNN from scratch (~79%).
- The model converged faster with fewer epochs.

Viva Questions with Answers

1. What is transfer learning?

Using a model trained on one task as a starting point for a different but related task.

2. Why use VGG16?

It's a proven architecture trained on a large dataset (ImageNet) and performs well on visual tasks.

3. What is the significance of freezing layers?

To retain pre-trained features and avoid retraining low-level filters.

Experiment 9 — Word Embeddings for IMDB Dataset

Aim: To implement and visualize word embeddings using the IMDB movie review dataset for sentiment classification.

Theory

Word embeddings map words into dense vectors in a continuous vector space, capturing semantic relationships. Unlike one-hot encoding, embeddings are low-dimensional and meaningful (e.g., "king" – "man" + "woman" $\approx$ "queen").

Implementation

Keras Implementation

```
from keras.datasets import imdb
from keras.models import Sequential
from keras.layers import Embedding, Flatten, Dense
from keras.preprocessing import sequence

max_features = 10000
max_len = 100

(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=max_features)
X_train = sequence.pad_sequences(X_train, maxlen=max_len)
X_test = sequence.pad_sequences(X_test, maxlen=max_len)

model = Sequential([
    Embedding(input_dim=max_features, output_dim=32, input_length=max_len),
    Flatten(),
    Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

history = model.fit(X_train, y_train, epochs=5, batch_size=128,
                    validation_split=0.2)
```

TensorFlow Implementation

```
import tensorflow as tf

max_features = 10000
max_len = 100

(X_train, y_train), (X_test, y_test) =
tf.keras.datasets.imdb.load_data(num_words=max_features)
X_train = tf.keras.preprocessing.sequence.pad_sequences(X_train, maxlen=max_len)
X_test = tf.keras.preprocessing.sequence.pad_sequences(X_test, maxlen=max_len)

model = tf.keras.Sequential([
    tf.keras.layers.Embedding(max_features, 32, input_length=max_len),
```

```

        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(1, activation='sigmoid')
    ])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

history = model.fit(X_train, y_train, epochs=5, batch_size=128,
                   validation_split=0.2)

```

PyTorch Implementation

```

import torch
import torch.nn as nn
from torchtext.datasets import IMDB
from torchtext.data.utils import get_tokenizer
from torchtext.vocab import build_vocab_from_iterator
from torch.nn.utils.rnn import pad_sequence
from torch.utils.data import DataLoader

# Preprocessing logic is more involved; here's a simplified embedding example
embedding = nn.Embedding(10000, 32)
input_example = torch.LongTensor([1, 15, 412, 78, 7])
embedded = embedding(input_example)
print("Embedded vector shape:", embedded.shape)

```

Graph: Training vs Validation Accuracy

```

import matplotlib.pyplot as plt

plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.title("Accuracy Over Epochs (Word Embedding)")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()

```

Sample Output

```

Epoch 1/5 - loss: 0.5184 - accuracy: 0.7351
Epoch 2/5 - loss: 0.3021 - accuracy: 0.8788
...
Final Training Accuracy: ~88%

```

Results

- Word embeddings helped the model reach ~88% accuracy.
- Dense representations improved learning compared to sparse one-hot encoding.
- The embedding layer learned semantic relationships.

Viva Questions with Answers

1. What is a word embedding?

A dense vector representation of words that captures semantic meaning.

2. How is it different from one-hot encoding?

One-hot is sparse and binary; embeddings are dense and semantic.

3. What does the Embedding layer do?

Maps each word index to a trainable dense vector.

4. Can embeddings be pretrained?

Yes, e.g., GloVe, Word2Vec, FastText.

5. Why is Flatten used after Embedding?

To convert the 2D sequence into a 1D vector for dense layers.

Experiment 10 — Implement a Recurrent Neural Network (RNN) for IMDB Movie Review Classification

Aim: To build a Recurrent Neural Network for binary sentiment classification on the IMDB movie review dataset.

Theory

RNNs are designed for sequence modeling tasks like text. They maintain a hidden state that updates with each word in the sequence, allowing information to persist over time. This makes them ideal for tasks where the order of input matters, such as sentiment analysis.

Implementation

Keras Implementation

```
from keras.datasets import imdb
from keras.models import Sequential
from keras.layers import Embedding, SimpleRNN, Dense
from keras.preprocessing.sequence import pad_sequences

max_features = 10000
maxlen = 100

(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=max_features)
X_train = pad_sequences(X_train, maxlen=maxlen)
X_test = pad_sequences(X_test, maxlen=maxlen)

model = Sequential([
    Embedding(max_features, 32),
    SimpleRNN(32),
    Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

history = model.fit(X_train, y_train, epochs=5, batch_size=128,
                    validation_split=0.2)
```

TensorFlow Implementation

```
import tensorflow as tf

max_features = 10000
maxlen = 100

(X_train, y_train), (X_test, y_test) =
tf.keras.datasets.imdb.load_data(num_words=max_features)
X_train = tf.keras.preprocessing.sequence.pad_sequences(X_train, maxlen=maxlen)
X_test = tf.keras.preprocessing.sequence.pad_sequences(X_test, maxlen=maxlen)

model = tf.keras.Sequential([
```

```

        tf.keras.layers.Embedding(max_features, 32),
        tf.keras.layers.SimpleRNN(32),
        tf.keras.layers.Dense(1, activation='sigmoid')
    ])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

history = model.fit(X_train, y_train, epochs=5, batch_size=128,
                   validation_split=0.2)

```

PyTorch Implementation

```

import torch
import torch.nn as nn
from torchtext.datasets import IMDB
from torchtext.data.utils import get_tokenizer
from torchtext.vocab import build_vocab_from_iterator

class RNNClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim):
        super(RNNClassifier, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.rnn = nn.RNN(embed_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.embedding(x)
        _, h_n = self.rnn(x)
        out = self.sigmoid(self.fc(h_n.squeeze(0)))
        return out

```

Graph: Training and Validation Accuracy

```

import matplotlib.pyplot as plt

plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title("RNN Accuracy on IMDB")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()

```

Sample Output

```

Epoch 1/5   157/157 [=====] - loss: 0.5402 - accuracy: 0.7172
Epoch 2/5   157/157 [=====] - loss: 0.3653 - accuracy: 0.8403
...

```

Final Accuracy: ~85% (Train), ~82% (Validation)

Results

- RNN captured sequence information in movie reviews.
- Achieved good accuracy (85%) with a simple architecture.
- Suitable for small to medium-length text sequences.

Viva Questions with Answers

1. What is an RNN?

A neural network designed for sequential data that maintains a memory of past inputs.

2. How is RNN different from regular feedforward networks?

RNNs share parameters across time steps and process inputs sequentially.

3. What is the limitation of RNNs?

They struggle with long-term dependencies due to vanishing gradients.

4. What are alternatives to Simple RNNs?

LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit).

5. What task did we solve with RNN in this experiment?

Binary sentiment classification of IMDB movie reviews.